Data Structure & Algorithms

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)

COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE

CONCEPT-FOCUSED LEARNING

MCQ-ORIENTED APPROACH

QUICK REVISION READY

COMPILED BY

Asabhya Maanav

2026 EDITION

VERSION 1.0

Table of Contents

Data Structure & Algorithms — Complete Notes

1. Introduction to Data Structures & Algorithm Analysis

1.1 Basic Concepts

Data, Data Item, Data Object and Data Type

Data is a collection of raw, unprocessed facts, figures and symbols that have no meaning by themselves.

A **Data Item** is a single unit of value; it is the smallest named unit of data (e.g., a single name or roll number).

A **Data Object** is a set/region of values of the same type (e.g., the set of all integers).

A **Data Type** is a collection of values together with a set of operations allowed on those values (e.g., `int`, `float`, `char`).

Data items are classified as **elementary** (cannot be divided, e.g., PIN code) and **group items** (can be subdivided, e.g., a Date into day, month, year).

Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a mathematical model of a data structure that specifies **WHAT** operations are performed but hides **HOW** they are implemented.

ADT separates the **logical (interface)** view from the **physical (implementation)** view — this is the principle of **information hiding / encapsulation**.

Classic ADT examples: **List, Stack, Queue, Tree, Graph, Set, Map**.

> **MCQ Trap:** ADT specifies behaviour (the *what*), NOT implementation (the *how*). A Stack ADT can be implemented by an array OR a linked list — both are valid.

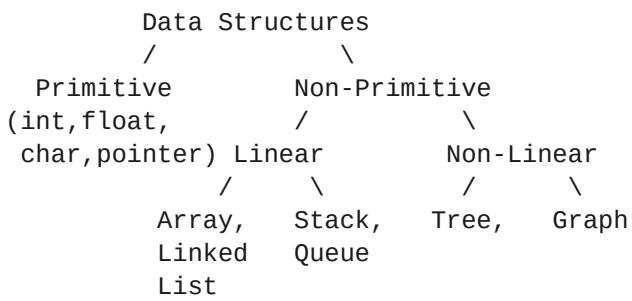
Data Structure — Definition and Need

A **Data Structure** is a particular way of **organizing, storing and retrieving** data in memory so that it can be used efficiently.

It defines the **logical relationship** between data elements plus the set of operations on them.

Need: efficient **storage**, fast **retrieval/search**, easy **modification**, and optimal use of **time** and **space**.

Classification of Data Structures



Basis	Type A	Type B
Structure	**Primitive** (int, char, float, pointer)	**Non-Primitive** (array, list, tree)
Arrangement	**Linear** (elements in sequence)	**Non-Linear** (hierarchical/network)
Memory	**Static** (size fixed at compile time, e.g., array)	**Dynamic** (size grows/shrinks at run time, e.g., linked list)

In a **Linear** data structure elements form a **sequence** and are traversed in a single run: **array, linked list, stack, queue**.

In a **Non-Linear** data structure an element may connect to **many** others (one-to-many / many-to-many): **tree, graph**.

> **Memory Trick:** "**SALL** are Linear" — **S**tack, **A**rray, **L**inked list, queue (**L**ine). Trees & Graphs are non-linear.

Operations on Data Structures

The core operations are **Traversal, Searching, Insertion, Deletion, Sorting, Merging and Updating**.

- **Traversal** — visiting each element exactly once.
- **Searching** — finding the location of a given element.
- **Insertion** — adding a new element.
- **Deletion** — removing an existing element.
- **Sorting** — arranging elements in ascending/descending order.
- **Merging** — combining two sorted structures into one.

Choosing the Right Data Structure

Selection depends on the **frequency of operations**, **time complexity** needed, **memory** available, and whether the data size is **known** in advance.

Example: frequent random access → **array**; frequent insert/delete in middle → **linked list**; LIFO order → **stack**; FIFO order → **queue**.

1.2 Algorithm Fundamentals

Definition and Characteristics

An **Algorithm** is a finite, step-by-step, unambiguous set of instructions to solve a particular problem.

The **5 characteristics** (by Donald Knuth) are:

- **Finiteness** — must terminate after a finite number of steps.
- **Definiteness** — each step must be precise and unambiguous.
- **Input** — zero or more externally supplied inputs.
- **Output** — at least one output is produced.
- **Effectiveness** — every operation must be basic enough to be carried out exactly in finite time.

> **Memory Trick:** "**FIDOE**" → **F**initeness, **I**nput, **D**efiniteness, **O**utput, **E**ffectiveness.

Algorithm vs Program vs Pseudocode

Aspect	Algorithm	Program	Pseudocode
Language	Natural / mathematical	Programming language	English-like + code structure
Execution	Cannot be executed directly	Executable on a machine	Not executable
Must terminate?	**Yes** (finiteness)	Not necessarily (e.g., OS loops)	Yes
Hardware-bound	No	Yes	No

Algorithm Design Techniques (Overview)

- **Divide & Conquer** — split → solve subproblems → combine (e.g., **Merge Sort, Quick Sort, Binary Search**).
- **Greedy** — pick the locally optimal choice each step (e.g., **Prim's, Kruskal's, Dijkstra's, Huffman coding**).
- **Dynamic Programming** — solve overlapping subproblems once and store results (e.g., **Floyd-Warshall, OBST, Bellman-Ford, LCS**).

- **Backtracking** — build solutions incrementally, abandon ("backtrack") on failure (e.g., **N-Queens, maze, Sudoku**).
- **Branch & Bound** — like backtracking but uses bounds to prune (e.g., **0/1 Knapsack, TSP**).

MCQ Trap: Greedy never reconsiders a choice, while **DP** considers all and reuses sub-results. Quick Sort & Merge Sort are **Divide & Conquer**, NOT greedy.

1.3 Asymptotic Notation & Complexity Analysis

Need for Complexity Analysis

Complexity analysis measures algorithm efficiency **independent of machine, compiler and language**, by counting growth of basic operations as input size **n** grows.

Time and Space Complexity

Time Complexity = amount of time taken as a function of input size **n**.

Space Complexity = total memory needed = **fixed part** (code, constants) + **variable part** (dynamic memory, recursion stack).

Best, Worst and Average Case

- **Best case** — minimum time; the most favourable input (e.g., target at first position in linear search).
- **Worst case** — maximum time; the least favourable input (e.g., target absent / at last position).
- **Average case** — expected time over all inputs (often hardest to compute).

MCQ Trap: We usually report the **Worst case** because it gives a guaranteed upper bound. Average case needs a probability distribution of inputs.

Asymptotic Notations

Big-O (O) — gives the **upper bound** (worst case). f(n) = O(g(n)) if there exist c > 0 and n0 such that **0 ≤ f(n) ≤ c·g(n)** for all n ≥ n0.

Big-Omega (Ω) — gives the **lower bound** (best case). f(n) = Ω(g(n)) if **0 ≤ c·g(n) ≤ f(n)** for all n ≥ n0.

Big-Theta (Θ) — gives the **tight bound** (both upper and lower). f(n) = Θ(g(n)) if **c1·g(n) ≤ f(n) ≤ c2·g(n)**.

Little-o (o) — strict upper bound that is **not tight**: f(n) = o(g(n)) means f grows strictly slower (e.g., n = o(n^2)).

Little-omega (ω) — strict lower bound, not tight: f grows strictly faster.

Notation	Bound	Intuition (comparison)
O	Upper (≤)	"less than or equal"
Ω	Lower (≥)	"greater than or equal"
Θ	Tight (=)	"equal"
o	Strict upper (<)	"strictly less than"
ω	Strict lower (>)	"strictly greater than"

Memory Trick: **O**mega = l**O**wer? NO! Big-**O** = upper, **O**mega = l**o**wer bound. Theta = Tight = "Two" sided.

Common Growth Rates (Ascending Order)

```
O(1) < O(log n) < O(sqrt n) < O(n) < O(n log n) < O(n^2)
     < O(n^3) < O(2^n) < O(n!) < O(n^n)
```

Complexity	Name	Example
O(1)	Constant	array index access, push/pop

Complexity	Name	Example
O(log n)	Logarithmic	**Binary Search**
O(n)	Linear	**Linear Search**
O(n log n)	Linearithmic	**Merge/Heap/Quick (avg) Sort**
O(n²)	Quadratic	**Bubble/Insertion/Selection Sort**
O(2■)	Exponential	recursive Fibonacci, subsets
O(n!)	Factorial	brute-force **TSP**, permutations

MCQ Trap: O(log n) < O(n) < O(n log n) — examiners love swapping `n log n` and `n²` ordering. n log n is always faster than n² for large n.

Rules for Finding Complexity

- A simple statement / assignment → **O(1)**.
- A single loop running n times → **O(n)**.
- Two nested loops (each n) → **O(n²)**.
- A loop that **halves** the range each step → **O(log n)**.
- **Consecutive** statements/loops → add, keep the **largest** (drop lower-order terms).
- **if-else** → take the branch with the **maximum** cost.
- Drop **constants** and **coefficients**: 3n² + 5n + 7 = **O(n²)**.

Analysis of Recursive Algorithms

A **recurrence relation** expresses a function in terms of its value on smaller inputs, e.g., Merge Sort: **T(n) = 2T(n/2) + O(n)**.

Methods to solve recurrences:

- **Substitution method** — guess the bound, prove by induction.
- **Recursion tree method** — draw the tree, sum the cost of all levels.
- **Master's Theorem** — direct formula for divide-and-conquer recurrences.

Master's Theorem

For a recurrence of the form **T(n) = aT(n/b) + f(n)** with **a ≥ 1, b > 1**, compare f(n) with **n^(log_b a)**:

- **Case 1:** if f(n) = O(n^(log_b a – ε)) → **T(n) = Θ(n^(log_b a))** (leaves dominate).
- **Case 2:** if f(n) = Θ(n^(log_b a)) → **T(n) = Θ(n^(log_b a) · log n)** (balanced).
- **Case 3:** if f(n) = Ω(n^(log_b a + ε)) with regularity condition → **T(n) = Θ(f(n))** (root dominates).

Recurrence	a, b, f(n)	Result
Merge Sort: T(n)=2T(n/2)+n	a=2,b=2	**Θ(n log n)** (Case 2)
Binary Search: T(n)=T(n/2)+1	a=1,b=2	**Θ(log n)** (Case 2)
T(n)=2T(n/2)+n²	a=2,b=2	**Θ(n²)** (Case 3)
T(n)=4T(n/2)+n	a=4,b=2	**Θ(n²)** (Case 1)

MCQ Trap: Master's Theorem does **NOT** apply when a is not constant, when f(n) is not polynomial vs n^(log_b a) (the "gap" cases), or for subtract-and-conquer recurrences like T(n)=T(n–1)+n.

Amortized Analysis

Amortized analysis gives the **average cost per operation over a worst-case sequence** of operations, not over a distribution of inputs.

Example: a **dynamic array** doubling on overflow has occasional **O(n)** resizes but **amortized O(1)** per insertion.

Three methods: **Aggregate, Accounting (banker's), and Potential** method.

Time–Space Trade-off

Often you can **save time by using more space** (e.g., **hashing**, lookup tables, memoization) or **save space at the cost of time** (e.g., recomputation).

> **Memory Trick:** "Time-Space trade-off" → **Hashing** trades **space** for **O(1)** time.

2. Arrays

2.1 Arrays — Fundamentals

Definition and Memory Representation

An **Array** is a collection of **homogeneous** (same data type) elements stored in **contiguous** memory locations, referenced by a common name and an **index/subscript**.

Arrays allow **random access** in **O(1)** using the index. Indexing typically starts at **0** (C/Java) or **1** (some languages).

1-D Array Address Calculation

For a 1-D array A with base address **B**, lower bound **LB**, and element size **w** bytes:

```
Address( A[i] ) = B + (i - LB) * w
```

Example: int A[10], base = 1000, w = 4, 0-based → Address(A[5]) = 1000 + (5–0)*4 = **1020**.

2-D Array Address Calculation

A 2-D array A[m][n] (m rows, n columns) can be stored in **Row-Major** or **Column-Major** order.

Row-Major (used by C, C++, Python): rows stored one after another.

```
Address( A[i][j] ) = B + [ (i - LB_r) * N_c + (j - LB_c) ] * w
   where N_c = number of columns
```

Column-Major (used by FORTRAN, MATLAB): columns stored one after another.

```
Address( A[i][j] ) = B + [ (j - LB_c) * N_r + (i - LB_r) ] * w
   where N_r = number of rows
```

> **MCQ Trap: C uses Row-Major; FORTRAN uses Column-Major.** In row-major you multiply the row index by the **number of columns**; examiners often swap "rows" and "columns" in the formula.

Multi-Dimensional Arrays

A 3-D array A[L][M][N] generalises the formula; total elements = **L × M × N**.

Operations and Their Complexity

Operation	Time Complexity
Access by index	**O(1)**
Search (unsorted)	**O(n)**
Search (sorted, binary)	**O(log n)**
Insertion at end	**O(1)**
Insertion at position (shift)	**O(n)**
Deletion at position (shift)	**O(n)**

Advantages and Limitations

Advantages: **O(1) random access**, **cache-friendly** (locality of reference), simple, no extra pointer overhead.

Limitations: **fixed size** (static), costly **insertion/deletion** (O(n) shifting), memory wastage if undersized, must be **contiguous**.

2.2 Special Arrays / Matrices

Sparse Matrix

A **Sparse Matrix** is a matrix in which **most elements are zero** (only few non-zero). Storing all elements wastes space.

Triplet / 3-tuple representation: store only non-zero values as rows of **(row, column, value)**. The first row stores **(#rows, #cols, #non-zero)**.

```
Matrix:                Triplet:
0 0 5                  Row Col Val
0 8 0       --->        3   3   3   <- (rows, cols, count)
0 0 0                   0   2   5
                        1   1   8
```

Linked representation: each non-zero stored as a node — saves space for very large sparse matrices and eases insertion.

Sparse Matrix Transpose

In the **transpose**, element (i, j) becomes (j, i); using the triplet form, swap row & column fields and re-sort. **Fast transpose** runs in **O(cols + non-zero)**.

Triangular, Diagonal and Tri-diagonal Matrices

- **Lower triangular** — all elements **above** the main diagonal are 0; non-zero count = **n(n+1)/2**.
- **Upper triangular** — all elements **below** the main diagonal are 0; non-zero count = **n(n+1)/2**.
- **Diagonal matrix** — only main diagonal non-zero; store **n** elements.
- **Tri-diagonal matrix** — non-zero only on main, one upper and one lower diagonal; **3n – 2** non-zero elements.

> **Memory Trick:** Triangular matrix needs **n(n+1)/2** cells; Tri-diagonal needs **3n – 2** cells.

3. Linked Lists

3.1 Concepts

Need for Linked List

A **Linked List** is a linear data structure where elements (**nodes**) are stored at **non-contiguous** memory locations and linked using **pointers**.

It overcomes array limitations: **dynamic size**, and **O(1)** insertion/deletion (given the node) without shifting.

Self-Referential Structure

Each node has two parts: **DATA** (the value) and **LINK/NEXT** (a pointer to the next node). A structure containing a pointer to its own type is **self-referential**.

```
HEAD
 |
 v
+----+----+   +----+----+   +----+------+
| 10 | *--+-->| 20 | *--+-->| 30 | NULL |
+----+----+   +----+----+   +----+------+
```

Array vs Linked List

Feature	Array	Linked List
Memory	**Contiguous**	**Non-contiguous**
Size	**Static** (fixed)	**Dynamic**
Access	**Random O(1)**	**Sequential O(n)**
Insert/Delete	**O(n)** (shifting)	**O(1)** (at known node)
Memory/element	No overhead	**Extra pointer** overhead
Locality	**Cache-friendly**	Poor locality

MCQ Trap: Arrays give **O(1) random access** but **O(n)** insertion; linked lists give **O(1)** insertion (at a node) but **O(n)** access. They are opposite trade-offs.

3.2 Types of Linked Lists

Singly Linked List (SLL)

Each node points only to the **next** node; the last node's link is **NULL**. Traversal is **one-directional** (forward only).

Doubly Linked List (DLL)

Each node has **3 parts**: **PREV pointer, DATA, NEXT pointer**. Allows **bidirectional** traversal; needs **extra memory** for the prev pointer.

```
NULL <-+----+----+----+-> <-+----+----+----+-> NULL
       |prev| 10 |next|      |prev| 20 |next|
       +----+----+----+      +----+----+----+
```

Circular Linked List (CLL)

The last node points back to the **first** node (no NULL). Can be **singly** or **doubly** circular. Useful for **round-robin** scheduling and circular buffers.

Header Linked List

A special **header node** at the front holds global information (e.g., count). Two kinds: **grounded** (last link NULL) and **circular** header list.

3.3 Operations on Linked Lists

Operation	SLL Time
Insert at beginning	**O(1)**
Insert at end	**O(n)** (O(1) with tail pointer)
Insert at position	**O(n)**
Delete at beginning	**O(1)**
Delete at end	**O(n)**
Search / Access	**O(n)**

Reversing a Linked List

Iterative reversal uses **3 pointers**: **prev, curr, next**, running in **O(n)** time and **O(1)** space. Recursive reversal uses **O(n)** stack space.

Floyd's Cycle Detection (Tortoise & Hare)

To detect a **loop**, use two pointers — **slow** (1 step) and **fast** (2 steps). If they ever meet, a loop exists. Runs in **O(n)** time, **O(1)** space.

```
slow ->  +1 each step
```

```
fast ->  +2 each step
If fast == slow  => LOOP exists
If fast == NULL  => NO loop
```

Finding Middle / Nth-from-End

The **middle** element is found by the slow/fast pointer technique in **one pass O(n)**. The **Nth node from end** is found by advancing one pointer N nodes ahead, then moving both together.

> **MCQ Trap:** In Floyd's algorithm the fast pointer moves **2** steps and slow moves **1**; they meet **inside** the loop, not necessarily at the loop's start.

3.4 Applications of Linked Lists

- **Polynomial representation** — each node stores **(coefficient, exponent)**; addition merges like terms.
- **Sparse matrix** representation.
- **Dynamic memory management** and **garbage collection** (free list).
- Implementation of **stacks, queues, graphs (adjacency list), hash table chaining**.

4. Stacks

4.1 Stack Fundamentals

Definition and LIFO Principle

A **Stack** is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle — the last element pushed is the first popped.

All insertions and deletions happen at **one end** called the **TOP**.

Stack ADT Operations

- **push(x)** — insert element x at top.
- **pop()** — remove and return the top element.
- **peek() / top()** — view the top element without removing.
- **isEmpty()** — true if stack has no elements.
- **isFull()** — true if stack (array) is full.

Overflow and Underflow

- **Overflow** — pushing onto a **full** stack (array implementation): condition **TOP == SIZE – 1**.
- **Underflow** — popping from an **empty** stack: condition **TOP == –1**.

```
Push order: 10,20,30        TOP -> | 30 |
                                   | 20 |
                                   | 10 |
                                   +----+

Pop returns 30 first (LIFO)
```

> **MCQ Trap:** Empty array stack has **TOP = –1**; full has **TOP = n – 1**. Underflow = pop on empty, Overflow = push on full.

4.2 Implementation

- **Array (static)** — fixed size, simple, but may overflow; push/pop are **O(1)**.
- **Linked list (dynamic)** — push/pop at head in **O(1)**, no fixed size, extra pointer memory.
- **Multiple stacks in one array** — two stacks can grow from **both ends** toward the middle for best space use.

4.3 Applications of Stack

Expression Notations

- **Infix** — operator **between** operands: `A + B`.
- **Prefix (Polish)** — operator **before** operands: `+ A B`.
- **Postfix (Reverse Polish)** — operator **after** operands: `A B +`.

Infix	Prefix	Postfix
A + B	**+AB**	**AB+**
A + B * C	**+A*BC**	**ABC*+**
(A+B)*C	***+ABC**	**AB+C***

Infix to Postfix Conversion

Scan left to right using an operator stack: output operands directly; pop operators of **higher/equal precedence** before pushing a new operator; `(` is pushed, `)` pops until `(`.

Evaluation of Postfix

Scan left to right; push operands; on an operator pop **two** operands, apply, push result. The final stack value is the answer. Runs in **O(n)**.

Operator precedence (high to low): **^ (right-assoc) > * / > + –**.

Other Stack Applications

- **Balancing/matching parentheses** and symbols.
- **Function calls & recursion** — the system uses an **activation record / stack frame** per call.
- **Reversing** a string or list.
- **Tower of Hanoi** — needs **2■ – 1** moves for n disks.
- **Backtracking** (maze, DFS), **undo** operations, **expression evaluation**.

> **Memory Trick:** Prefix = operator **Pre** (before); Postfix = operator **Post** (after). To convert infix→prefix, reverse, convert to postfix logic, reverse again.

5. Queues

5.1 Queue Fundamentals

Definition and FIFO Principle

A **Queue** is a linear data structure following **FIFO (First-In-First-Out)** — the first element inserted is the first removed.

Insertion happens at the **REAR** (enqueue); deletion happens at the **FRONT** (dequeue).

Queue ADT Operations

- **enqueue(x)** — add x at rear.
- **dequeue()** — remove element from front.
- **front()**, **rear()** — peek front/rear.
- **isEmpty()**, **isFull()**.

```
FRONT                          REAR
  |                              |
  v                              v
[ 10 ][ 20 ][ 30 ][ 40 ]
dequeue removes 10 (first in)
```

Overflow / Underflow

In a simple **linear array queue**, **REAR == SIZE – 1** signals overflow even if front cells are free — causing **false overflow** (wastage). This is why **circular queues** exist.

5.2 Types of Queues

Linear Queue

Simple array/linked queue; suffers from **false overflow** because dequeued front cells cannot be reused.

Circular Queue

The rear wraps around to the front using **modulo** arithmetic: `rear = (rear + 1) % SIZE`. This reuses freed slots.

- **Empty** condition: **FRONT == –1**.
- **Full** condition: **(REAR + 1) % SIZE == FRONT**.

```
        [0][1][2][3][4]
rear ----------^   front pointer wraps via % SIZE
```

Double-Ended Queue (Deque)

A **Deque** allows insertion and deletion at **both ends**. Variants:

- **Input-restricted deque** — insertion only at **one** end, deletion at both.
- **Output-restricted deque** — deletion only at **one** end, insertion at both.

Priority Queue

Each element has a **priority**; the element with **highest priority** is served first (ties → FIFO). Two kinds: **ascending** and **descending** priority queue. Best implemented with a **heap** (insert/delete **O(log n)**).

> **MCQ Trap:** A circular queue is **full** when **(rear+1)%size == front**; one slot is intentionally left empty in the classic implementation to distinguish full from empty.

5.3 Applications of Queue

- **CPU and disk scheduling** (ready queue).
- **Breadth-First Search (BFS)** in graphs/trees.
- **Buffering**, **spooling** (printer queue), **interrupt handling**.
- Implementing a **stack using 2 queues**, and a **queue using 2 stacks**.

> **Memory Trick: Stack = LIFO** (like a stack of plates); **Queue = FIFO** (like a line at a counter).

6. Trees

6.1 Tree Terminology

A **Tree** is a **non-linear, hierarchical** data structure of nodes with **one root** and no cycles; a tree with **n** nodes has exactly **n – 1** edges.

- **Root** — the topmost node (no parent).
- **Edge** — link between a parent and child.
- **Parent / Child** — immediate predecessor / successor.
- **Sibling** — nodes with the same parent.
- **Leaf (external)** — node with **no children** (degree 0).
- **Internal node** — node with at least one child.
- **Degree of a node** — number of its children; **degree of tree** — max degree of any node.
- **Level** — root is level **0** (or 1 by some books); increases by 1 per edge downward.

- **Height/Depth** — height of a node = longest path (edges) to a leaf; **height of tree** = height of root. **Depth** of a node = edges from root to that node.
- **Path, Ancestor, Descendant, Subtree, Forest** (a forest = a set of disjoint trees).

MCQ Trap: A tree with n nodes has exactly **n – 1 edges**. **Depth** is measured from the root downward; **height** is measured from the node down to its deepest leaf.

6.2 Binary Trees

Definition and Properties

A **Binary Tree** is a tree in which every node has **at most 2 children** (left and right).

Key properties (root at level 0, height h = edges):

- Maximum nodes at level **l** = **2^l**.
- Maximum nodes in a binary tree of height **h** = **2^(h+1) – 1**.
- Minimum height of a binary tree with **n** nodes = **⌊log₂ n⌋**.
- In any non-empty binary tree, number of **leaf nodes** L = number of nodes with **2 children** + 1, i.e., **L = n₂ + 1**.
- A binary tree with **n** nodes has **n + 1** NULL (empty) links.

Types of Binary Trees

Type	Definition
Full / Strictly / Proper	Every node has **0 or 2** children
Complete	All levels filled except possibly the last, filled **left to right**
Perfect	All internal nodes have 2 children AND all leaves at the **same level**; has **2^(h+1)–1** nodes
Skewed	Every node has only one child (**left-skewed** or **right-skewed**) — behaves like a list
Balanced	Height difference of subtrees is bounded (e.g., **AVL**)
Extended (2-tree)	NULL links replaced by special **external** nodes

MCQ Trap: A **Complete** binary tree fills the last level left-to-right; a **Full** binary tree allows the last level partially filled as long as every node has 0 or 2 children. A **Perfect** tree is both full and complete with all leaves at the same depth.

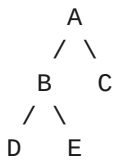
Representation of Binary Trees

- **Array (sequential):** for node at index i (1-based): left child = **2i**, right child = **2i + 1**, parent = **⌊i/2⌋**. Wastes space for skewed trees.
- **Linked representation:** each node = **(left pointer, data, right pointer)**. Preferred for general trees.

6.3 Tree Traversals

Depth-First Traversals (recursive)

- **Inorder (LNR):** Left → Root → Right. For a **BST**, inorder gives **sorted ascending** order.
- **Preorder (NLR):** Root → Left → Right. Used to **copy/serialize** a tree; first node is the **root**.
- **Postorder (LRN):** Left → Right → Root. Used to **delete/free** a tree; last node is the **root**.




```
Inorder  : D B E A C
Preorder : A B D E C
```

```
Postorder: D E B C A
Level    : A B C D E
```

Level-Order Traversal (BFS)

Visits nodes **level by level** using a **queue**; runs in **O(n)**.

Constructing a Tree from Traversals

A unique binary tree can be built from **Inorder + Preorder** OR **Inorder + Postorder**. **Preorder + Postorder alone is NOT enough** (except for full binary trees).

> **MCQ Trap: Inorder is mandatory** to reconstruct a unique binary tree. Preorder's first node and Postorder's last node give the **root**.

6.4 Binary Search Tree (BST)

Properties

A **BST** is a binary tree where for every node: **left subtree < node < right subtree**, and both subtrees are BSTs (no duplicates in the classic definition).

Operations and Complexity

Operation	Best/Avg	Worst (skewed)
Search	**O(log n)**	**O(n)**
Insert	**O(log n)**	**O(n)**
Delete	**O(log n)**	**O(n)**

All BST operations are **O(h)** where h is the height; balanced → **O(log n)**, skewed → **O(n)**.

Deletion — 3 Cases

- **Leaf node** — simply remove it.
- **One child** — replace node with its single child.
- **Two children** — replace with **inorder successor** (smallest in right subtree) or **inorder predecessor** (largest in left subtree), then delete that node.

> **MCQ Trap: Inorder traversal of a BST yields a sorted (ascending) sequence.** Worst-case BST height is **O(n)** when keys are inserted in sorted order (degenerates into a skewed list).

6.5 Balanced and Special Trees

AVL Tree

An **AVL Tree** (Adelson-Velsky & Landis, **1962**) is a **self-balancing BST** where for every node the **Balance Factor (BF) = height(left) − height(right) ∈ {−1, 0, +1}**.

When |BF| > 1 after insertion, one of **4 rotations** restores balance:

- **LL (Left-Left)** → single **right** rotation.
- **RR (Right-Right)** → single **left** rotation.
- **LR (Left-Right)** → **left** then **right** rotation (double).
- **RL (Right-Left)** → **right** then **left** rotation (double).

All AVL operations (search/insert/delete) are guaranteed **O(log n)** because height is kept ≈ **1.44 log■ n**.

```
Insert causing LL imbalance      After Right Rotation
        30 (BF=+2)                       20
        /                               /  \
      20                              10    30
      /
    10
```

MCQ Trap: Balance factor = **height(left) – height(right)**. LL & RR need **single** rotations; LR & RL need **double** rotations. AVL is stricter (more balanced) than a Red-Black tree.

B-Tree

A **B-Tree of order m** is a balanced **multi-way search tree** used for **disk-based** storage (databases, file systems):

- Every node has at most **m children** and at most **m – 1 keys**.
- Every non-root node has at least **⌈m/2⌉ children** (i.e., **⌈m/2⌉ – 1** keys).
- All **leaves appear at the same level** (perfectly height-balanced).
- Height is **O(log n)**; search/insert/delete are **O(log n)**.

B+ Tree

A **B+ Tree** stores **all data/keys in leaf nodes**; internal nodes store only **keys for routing**. Leaves are linked in a **doubly linked list** for fast **range/sequential** queries.

MCQ Trap: In a **B+ tree** data is stored **only in leaves** and leaves are linked, giving efficient range queries. In a **B-tree** data can be in internal nodes too. B/B+ trees minimise **disk I/O**.

Red-Black Tree

A **Red-Black Tree** is a self-balancing BST with **5 properties**: (1) every node is **red or black**, (2) the **root is black**, (3) all **leaves (NIL) are black**, (4) a red node's children are **black** (no two reds in a row), (5) every root-to-leaf path has the **same number of black nodes** (black-height). Operations are **O(log n)**.

Threaded Binary Tree

A **Threaded Binary Tree** replaces NULL pointers with **threads** pointing to the inorder **predecessor/successor**, enabling **traversal without recursion or a stack**. Types: **one-way (single)** and **two-way (double)** threaded.

Expression Tree

An **Expression Tree** stores an arithmetic expression: **leaves = operands**, **internal nodes = operators**. Its **inorder** gives infix, **preorder** gives prefix, **postorder** gives postfix.

6.6 Optimal Binary Search Tree (OBST)

An **OBST** is a BST built to minimise the **total expected search cost** when keys have different **access frequencies/probabilities**.

It is solved with **Dynamic Programming**: minimise **Σ (frequency × (level of key))**. Construction runs in **$O(n^3)$** (or **$O(n^2)$** with Knuth's optimisation).

The cost recurrence: **C[i][j] = min over r (C[i][r–1] + C[r+1][j]) + Σ frequencies**.

MCQ Trap: OBST does **NOT** minimise tree height — it minimises **weighted (expected) search cost**. Frequently accessed keys are placed **closer to the root**.

6.7 Lemmas & Theorems on Trees

- A binary tree of height **h** has at most **2^(h+1) – 1** nodes and at least **h + 1** nodes.
- The number of binary trees possible with **n** nodes (unlabelled) = the **n-th Catalan number** = **C(2n, n)/(n + 1)**.
- With **n labelled** nodes, number of trees = **Catalan(n) × n!**.
- A complete binary tree with n nodes has height **⌊log⌋ n⌋**.
- Number of leaf nodes in a full binary tree with **i** internal nodes = **i + 1**.

Memory Trick: Number of distinct binary tree shapes with n nodes = **Catalan(n)**: 1, 1, 2, 5, 14, 42 for n = 0,1,2,3,4,5.

7. Heaps & Heapsort

7.1 Heap Concepts

A **Heap** is a **complete binary tree** that satisfies the **heap property** and is stored in an **array**.

- **Max-Heap** — every parent ≥ its children; the **maximum** is at the **root**.
- **Min-Heap** — every parent ≤ its children; the **minimum** is at the **root**.

Array index formulas (1-based): parent(i) = ⌊i/2⌋, left(i) = **2i**, right(i) = **2i + 1**. (0-based: left = **2i+1**, right = **2i+2**, parent = ⌊(i−1)/2⌋.)

```
Max-Heap:          Array (1-based):
        50          [50, 30, 40, 10, 20]
       /  \
      30   40
     / \
    10  20
```

MCQ Trap: A heap is a **complete** binary tree (NOT a BST) — there is **no** left<right ordering, only parent–child ordering. Inorder of a heap is **not** sorted.

7.2 Heap Operations

- **Insertion** — add at the end, then **heapify-up (percolate/sift-up)**: **O(log n)**.
- **Delete root (extract max/min)** — replace root with last element, then **heapify-down (sift-down)**: **O(log n)**.
- **Build-Heap** — converting an arbitrary array into a heap by sifting down from the last internal node; runs in **O(n)** (a tight bound, not O(n log n)).
- **getMax / getMin** (peek) — **O(1)**.

MCQ Trap: Build-Heap is O(n), not O(n log n) — examiners commonly state it wrongly. A single insertion is O(log n).

7.3 Heapsort

Heapsort steps: (1) **Build a Max-Heap** from the array; (2) repeatedly **swap root with last element**, reduce heap size, and **heapify-down**.

- Time complexity: **O(n log n)** in **best, average and worst** cases.
- Space: **O(1)** — it is **in-place**.
- Stability: **NOT stable**.

A **Priority Queue** is most efficiently implemented with a heap: **insert** and **extract** in **O(log n)**, **peek** in **O(1)**.

Memory Trick: Heapsort = Build-Heap **O(n)** + n deletions each **O(log n)** = **O(n log n)**, in-place, unstable.

8. Graphs

8.1 Graph Terminology

A **Graph G = (V, E)** is a non-linear structure of a set of **vertices V** and a set of **edges E** connecting them.

- **Directed graph (digraph)** — edges have direction; **Undirected** — edges are bidirectional.
- **Weighted graph** — edges carry weights/costs.
- **Complete graph** — every pair connected; **undirected complete** has **n(n−1)/2** edges, **directed** has **n(n−1)**.
- **Degree** — number of incident edges; in a digraph: **in-degree** and **out-degree**. Sum of all degrees = **2 × |E|** (Handshaking Lemma).

- **Path, Cycle, Loop (self-edge), Multigraph (parallel edges), Subgraph**.
- **Connected** (undirected, path between all pairs); **Strongly connected** (digraph, directed path both ways); **Weakly connected**.

MCQ Trap: Max edges in an **undirected** simple graph = **n(n–1)/2**; in a **directed** simple graph = **n(n–1)**. Sum of degrees = **2|E|** (handshaking).

8.2 Graph Representation

Representation	Space	Edge check	Best for
Adjacency Matrix	**O(V²)**	**O(1)**	Dense graphs
Adjacency List	**O(V + E)**	**O(degree)**	Sparse graphs
Incidence Matrix	**O(V × E)**	—	Edge-centric problems

```
Graph: 1--2, 1--3, 2--3

Adjacency Matrix:     Adjacency List:
   1 2 3              1 -> 2 -> 3
1 [0 1 1]             2 -> 1 -> 3
2 [1 0 1]             3 -> 1 -> 2
3 [1 1 0]
```

MCQ Trap: Adjacency **matrix** uses **O(V²)** space regardless of edges; adjacency **list** uses **O(V+E)** and is preferred for **sparse** graphs.

8.3 Graph Traversal

Breadth-First Search (BFS)

BFS explores level by level using a **QUEUE**; time **O(V + E)** (list) / **O(V²)** (matrix). Finds the **shortest path in an unweighted graph**.

Depth-First Search (DFS)

DFS explores as deep as possible using a **STACK / recursion**; time **O(V + E)**. Used for **cycle detection, topological sort, connected components, strongly connected components**.

```
Graph:  A-B, A-C, B-D
BFS from A: A B C D   (queue)
DFS from A: A B D C   (stack/recursion)
```

MCQ Trap: BFS uses a Queue, **DFS uses a Stack** (or recursion). BFS gives shortest path in **unweighted** graphs; DFS does not.

8.4 Minimum Spanning Tree (MST)

A **Spanning Tree** of a connected graph with n vertices has exactly **n – 1 edges** and **no cycle**. A **Minimum Spanning Tree (MST)** has the **minimum total edge weight**.

Prim's Algorithm

Prim's grows the MST from a **start vertex**, repeatedly adding the **minimum-weight edge** connecting a tree vertex to a non-tree vertex (**greedy**).

- Time: **O(V²)** with adjacency matrix; **O(E log V)** with a binary heap.

Kruskal's Algorithm

Kruskal's sorts all edges ascending and adds the next smallest edge that **does not form a cycle**, using **Union-Find (Disjoint Set)**.

- Time: **O(E log E) = O(E log V)** (dominated by sorting).

Aspect	Prim's	Kruskal's
Approach	Grows one tree from a vertex	Picks global minimum edges
Data structure	Priority queue / matrix	**Union-Find** + sorting
Best for	**Dense** graphs	**Sparse** graphs
Time	**O(E log V)** / $O(V^2)$	**O(E log E)**

MCQ Trap: Both Prim's and Kruskal's are **Greedy**. A graph may have **multiple MSTs** if edge weights repeat, but the **total weight is unique**. MST has exactly **V – 1 edges**.

8.5 Shortest Path Algorithms

Dijkstra's Algorithm

Dijkstra's finds **single-source shortest paths** in a graph with **non-negative** weights (**greedy**).

- Time: **$O(V^2)$** (array) or **O((V + E) log V)** (binary heap) or **O(E + V log V)** (Fibonacci heap).
- **Fails with negative edges.**

Bellman-Ford Algorithm

Bellman-Ford finds single-source shortest paths and **handles negative edge weights**; it also **detects negative cycles**. Uses **Dynamic Programming** (relax all edges **V – 1** times).

- Time: **O(V × E)**.

Floyd-Warshall Algorithm

Floyd-Warshall finds **All-Pairs Shortest Paths** using **Dynamic Programming**; handles negative edges (no negative cycles).

- Time: **$O(V^3)$**, Space: **$O(V^2)$**.

Warshall's Algorithm

Warshall's algorithm computes the **transitive closure** (reachability) of a graph in **$O(V^3)$**.

Algorithm	Problem	Negative edges?	Time
BFS	Unweighted shortest path	n/a	**O(V+E)**
Dijkstra	Single-source	**No**	**O((V+E)log V)**
Bellman-Ford	Single-source	**Yes**	**O(VE)**
Floyd-Warshall	All-pairs	Yes (no neg cycle)	**$O(V^3)$**

MCQ Trap: Dijkstra fails with negative weights; use **Bellman-Ford**. Floyd-Warshall is **all-pairs $O(V^3)$** and is a **DP** algorithm, while Dijkstra/Prim are **greedy**.

8.6 Other Graph Topics

- **Topological Sort** — linear ordering of a **DAG** (Directed Acyclic Graph) such that every edge u→v has u before v; done via **DFS** or **Kahn's (BFS in-degree)** algorithm in **O(V + E)**.
- **Disjoint Set / Union-Find** — supports **Find** and **Union**; with **union by rank** + **path compression**, near-constant **$O(\alpha(n))$** (inverse Ackermann) amortised time. Used in **Kruskal's**.
- **Articulation Point / Bridge** — a vertex/edge whose removal **disconnects** the graph; found via DFS in **O(V + E)**.

MCQ Trap: Topological sort exists only for a DAG (no cycles). Union-Find with path compression + union by rank gives **almost O(1)** amortised operations.

9. Searching

9.1 Searching Algorithms

Linear / Sequential Search

Checks each element one by one until found or list ends. Works on **unsorted** data.

- Best **O(1)**, Average/Worst **O(n)**, Space **O(1)**.
- **Sentinel** version places the key at the end to remove the bound check, reducing comparisons.

Binary Search

Works **only on a SORTED array**; repeatedly halves the search range by comparing with the **middle** element.

- Best **O(1)**, Average/Worst **O(log n)**, Space **O(1)** iterative / **O(log n)** recursive.
- Mid calculation: use `mid = low + (high – low)/2` to **avoid integer overflow**.

```
Search 23 in [2,5,8,12,16,23,38,56]
low=0 high=7 mid=3 (12)  23>12 -> right
low=4 high=7 mid=5 (23)  FOUND at index 5
```

Interpolation Search

Improves binary search on **uniformly distributed sorted** data by estimating the probable position.

- Average **O(log log n)**, Worst **O(n)** (non-uniform data).

Fibonacci Search

Uses **Fibonacci numbers** to divide the sorted array; avoids division (only +/–), useful when division is costly.

- Time **O(log n)**.

Search	Data needs	Best	Avg	Worst
Linear	Unsorted OK	**O(1)**	**O(n)**	**O(n)**
Binary	**Sorted**	**O(1)**	**O(log n)**	**O(log n)**
Interpolation	Sorted+uniform	**O(1)**	**O(log log n)**	**O(n)**
Fibonacci	**Sorted**	**O(1)**	**O(log n)**	**O(log n)**

MCQ Trap: Binary search requires sorted data; on unsorted data it fails. Interpolation search is **O(log log n)** on uniform data but degrades to **O(n)**.

10. Sorting & Merging

10.1 Sorting Fundamentals

- **Internal sort** — all data fits in **main memory**; **External sort** — data on **disk/tape** (e.g., External Merge Sort).
- **Stable sort** — preserves relative order of equal keys (**Bubble, Insertion, Merge, Counting, Radix**).
- **Unstable sort** — may reorder equal keys (**Selection, Quick, Heap, Shell**).
- **In-place sort** — uses **O(1)** extra space (**Bubble, Insertion, Selection, Heap, Quick**).
- **Comparison-based** (Bubble, Quick, Merge, Heap) vs **Non-comparison** (Counting, Radix, Bucket).

Memory Trick (Stable sorts): "**MIB-RC**" → **M**erge, **I**nsertion, **B**ubble, **R**adix, **C**ounting are stable. **Q**uick, **H**eap, **S**election are NOT (**QHS**).

10.2 Sorting Algorithms

Bubble Sort

Repeatedly swaps adjacent out-of-order elements; largest "bubbles" to the end each pass. Optimised (with a swap flag) gives best case **O(n)** on sorted input.

Insertion Sort

Builds the sorted part one element at a time by **inserting** each into its correct position. Excellent for **small or nearly-sorted** data; best **O(n)**. Stable, in-place, **online**.

Selection Sort

Repeatedly selects the **minimum** from the unsorted part and places it next. Always **O(n²)** comparisons (n(n–1)/2) but only **O(n) swaps** — fewest swaps among the simple sorts. Unstable.

Quick Sort

Divide & Conquer: pick a **pivot**, **partition** elements (< pivot | > pivot), recurse.

- Best/Average **O(n log n)**; **Worst O(n²)** (already sorted with first/last pivot).
- Space **O(log n)** (recursion stack); **in-place**, **unstable**.
- Worst case avoided by **randomized** or **median-of-three** pivot.

Merge Sort

Divide & Conquer: split in half, sort each half, **merge**.

- Best/Average/Worst all **O(n log n)** — guaranteed.
- Space **O(n)** (not in-place); **stable**. Ideal for **linked lists** and **external sorting**.

Heap Sort

Build a max-heap then extract the max n times.

- Best/Average/Worst **O(n log n)**; Space **O(1)** in-place; **unstable**.

Shell Sort

Generalised insertion sort using a **gap sequence** (diminishing increments); reduces large shifts.

- Best **O(n log n)**; Worst **O(n²)** or **O(n^1.5)** depending on the gap sequence; in-place, unstable.

Radix Sort

Non-comparison sort processing digits from **LSD (least significant)** to MSD; uses a stable sub-sort (counting) per digit.

- Time **O(d × (n + b))** where d = digits, b = base (radix); **stable**, not in-place.

Counting Sort

Non-comparison sort that counts occurrences of each key; works for a **small integer range k**.

- Time **O(n + k)**; Space **O(k)**; **stable**.

Bucket / Bin Sort

Distributes elements into **buckets**, sorts each bucket, then concatenates.

- Average **O(n + k)**; Worst **O(n²)** (all in one bucket); stable if the bucket sort is stable.

10.3 Merging

- **Merging two sorted lists** of sizes m and n into one sorted list takes **O(m + n)** time using two pointers.
- **k-way merge** combines k sorted lists, efficiently done with a **min-heap** in **O(N log k)**.
- **External Merge Sort** sorts data too large for memory by sorting runs then k-way merging from disk.

10.4 Complexity of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble	**O(n)**	O(n²)	**O(n²)**	O(1)	**Yes**
Insertion	**O(n)**	O(n²)	**O(n²)**	O(1)	**Yes**
Selection	O(n²)	O(n²)	**O(n²)**	O(1)	**No**
Quick	O(n log n)	**O(n log n)**	**O(n²)**	O(log n)	**No**
Merge	O(n log n)	O(n log n)	**O(n log n)**	**O(n)**	**Yes**
Heap	O(n log n)	O(n log n)	**O(n log n)**	O(1)	**No**
Shell	O(n log n)	O(n^1.25)	**O(n²)**	O(1)	**No**
Counting	O(n+k)	O(n+k)	**O(n+k)**	O(k)	**Yes**
Radix	O(d(n+b))	O(d(n+b))	**O(d(n+b))**	O(n+b)	**Yes**
Bucket	O(n+k)	O(n+k)	**O(n²)**	O(n)	**Yes**

Lower bound for any **comparison-based** sort = **Ω(n log n)** (proved via the decision-tree model). Non-comparison sorts (Counting, Radix, Bucket) beat this because they don't compare elements.

MCQ Trap: Quick Sort worst case is O(n²) (not O(n log n)) but it is usually fastest in practice. **Merge Sort is always O(n log n)** and **stable** but needs **O(n)** extra space. Heap Sort is O(n log n) **in-place** but **unstable**.

Memory Trick: "**Quick** is **quick** on average but **slow (n²)** when sorted." Use **Insertion** for nearly-sorted/small data; **Merge** for linked lists/stability; **Heap** for guaranteed O(n log n) in-place.

11. Hashing

11.1 Hashing Concepts

Hashing maps a key to an index of a **hash table** using a **hash function**, giving **average O(1)** insert/search/delete.

- **Hash table** — array of size **m** storing keys/records.
- **Hash function h(k)** — maps key k to index in **[0, m–1]**.
- **Load factor (α) = n / m** (n = items, m = slots); performance degrades as α → 1.

A **good hash function** is **fast to compute**, **uniformly distributes** keys, and **minimises collisions**.

11.2 Hash Functions

- **Division method: h(k) = k mod m**; choose **m prime** (avoid powers of 2/10).
- **Mid-square method:** square the key and take **middle digits**.
- **Folding method:** split the key into parts and **add** them.
- **Multiplication method: h(k) = ⌊m (kA mod 1)⌋** with constant **A ≈ 0.6180339 (Knuth)**.

MCQ Trap: In the **division method**, the table size **m should be a prime** number (not a power of 2) for good distribution.

11.3 Collision and Collision Resolution

A **collision** occurs when two keys hash to the **same index**.

Open Hashing — Separate Chaining

Each slot holds a **linked list** of all keys hashing there. Simple, table can exceed size; search time = **O(1 + α)**.

Closed Hashing — Open Addressing

All elements stored **in the table itself**; on collision, **probe** for the next free slot:

- **Linear Probing:** h(k, i) = (h(k) + i) mod m → suffers **primary clustering**.
- **Quadratic Probing:** h(k, i) = (h(k) + c■i + c■i²) mod m → suffers **secondary clustering**.
- **Double Hashing:** h(k, i) = (h■(k) + i·h■(k)) mod m → **best**, least clustering.

Rehashing — when load factor is high, build a **bigger table** and reinsert all keys.

```
Insert 18,18 collide -> probe:
Linear:    next slot (i+1)
Quadratic: i+1, i+4, i+9 ...
Double:    step = h2(k)
```

Method	Storage	Clustering
Separate Chaining	Linked lists outside	None (chains)
Linear Probing	In table	**Primary**
Quadratic Probing	In table	**Secondary**
Double Hashing	In table	**Minimal**

> **MCQ Trap: Linear probing → primary clustering; Quadratic probing → secondary clustering; Double hashing** minimises both and is the best open-addressing scheme.

12. Programming Constructs (PASCAL & General)

12.1 PASCAL / Procedural Concepts

A **PASCAL** program structure: `program name; ... begin ... end.` (note the **final period**). It is a **strongly-typed**, structured language by **Niklaus Wirth (1970)**.

- **Data types:** integer, real, char, boolean; user-defined and **records**.
- **Variables/constants** declared in `var` and `const` sections.

Looping Constructs

- **for** — counter-controlled loop with a known count.
- **while ... do** — entry-controlled (pre-test); may run **0** times.
- **repeat ... until** — exit-controlled (post-test); runs **at least once**.

> **MCQ Trap:** `while` is **entry-controlled** (condition checked first → may execute zero times); `repeat-until` is **exit-controlled** (executes **at least once**).

Break, Continue

break exits the loop immediately; **continue** skips to the next iteration. (Standard PASCAL uses `Break`/`Continue` in later dialects.)

Functions and Procedures

- A **function** returns a value; a **procedure** does not.
- **Parameter passing: call by value** (copy, original unchanged) vs **call by reference** (`var` parameter, original modified).

> **MCQ Trap: Call by value** copies arguments (callee changes don't affect caller); **call by reference** (PASCAL `var`) lets the callee modify the caller's variable.

Recursion, Arrays, Records

- **Recursion** — a procedure/function calling itself; needs a **base case** to terminate; uses the **system stack** (activation records).

- **Arrays** — `array[1..n] of integer`; **records** group heterogeneous fields (like a struct).

13. Quick Revision — Exam Pointers

13.1 Key Complexities Cheat-Sheet

Structure / Op	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Sorted Array	O(1)	**O(log n)**	O(n)	O(n)
Linked List	O(n)	O(n)	**O(1)**	**O(1)**
Stack/Queue	O(n)	O(n)	**O(1)**	**O(1)**
BST (avg)	O(log n)	O(log n)	O(log n)	O(log n)
BST (worst)	O(n)	O(n)	O(n)	O(n)
AVL / RB Tree	**O(log n)**	O(log n)	O(log n)	O(log n)
Hash Table (avg)	—	**O(1)**	O(1)	O(1)
Heap	O(1) peek	O(n)	**O(log n)**	O(log n)
B/B+ Tree	—	O(log n)	O(log n)	O(log n)

13.2 Must-Remember Formulas

- Tree with n nodes → **n – 1 edges**; binary tree of height h → max **2^(h+1) – 1** nodes.
- Leaf nodes in binary tree: **L = n■ + 1**.
- Complete undirected graph edges: **n(n–1)/2**; directed: **n(n–1)**.
- Tower of Hanoi moves: **2■ – 1**.
- Comparison-sort lower bound: **Ω(n log n)**.
- Heap index (1-based): left **2i**, right **2i+1**, parent **■i/2■**.
- Binary trees with n nodes = **Catalan(n) = C(2n,n)/(n+1)**.

Memory Trick: Greedy → **Prim, Kruskal, Dijkstra, Huffman**. DP → **Floyd-Warshall, Bellman-Ford, OBST, LCS, Matrix-chain**. Divide & Conquer → **Merge sort, Quick sort, Binary search**.

Shortcut: Negative edges → **Bellman-Ford**; all-pairs → **Floyd-Warshall (O(V³))**; unweighted shortest path → **BFS**; non-negative single source → **Dijkstra**.